

ProductMind — Agentic RAG System for Product Intelligence

A Career-Transition Portfolio Project | June 2026

Why This Project (Context for Kiruthika)

You have a genuine edge most AI candidates don't: you've worked inside a real product-review analytics pipeline at Capchek — normalising buyer data, defining trust KPIs, building dashboards that tell brands *why* customers are unhappy. Most people build RAG demos around random PDFs. You're going to build one that solves a problem you've lived.

The story you'll tell in interviews:

"I come from product analytics — I was already working with review data, KPIs, support tickets. I built ProductMind to add an intelligence layer on top of that: an agent that can answer product questions from ingested documentation, escalate to web search when the knowledge base falls short, and evaluate its own answers. The architecture maps directly to production multi-agent systems — supervisor orchestrating specialised workers, with full tracing and evaluation."

That's a career-change narrative that lands.

Project Overview

Build a **multi-agent RAG system** that ingests product documentation (PDFs, URLs) into a vector database, then answers queries through a **supervisor-worker architecture** — routing to RAG retrieval or external tools based on context relevance, with automatic LLM fallback, evaluation scoring, and full step-by-step tracing.

Realistic Timeline: 6 Days

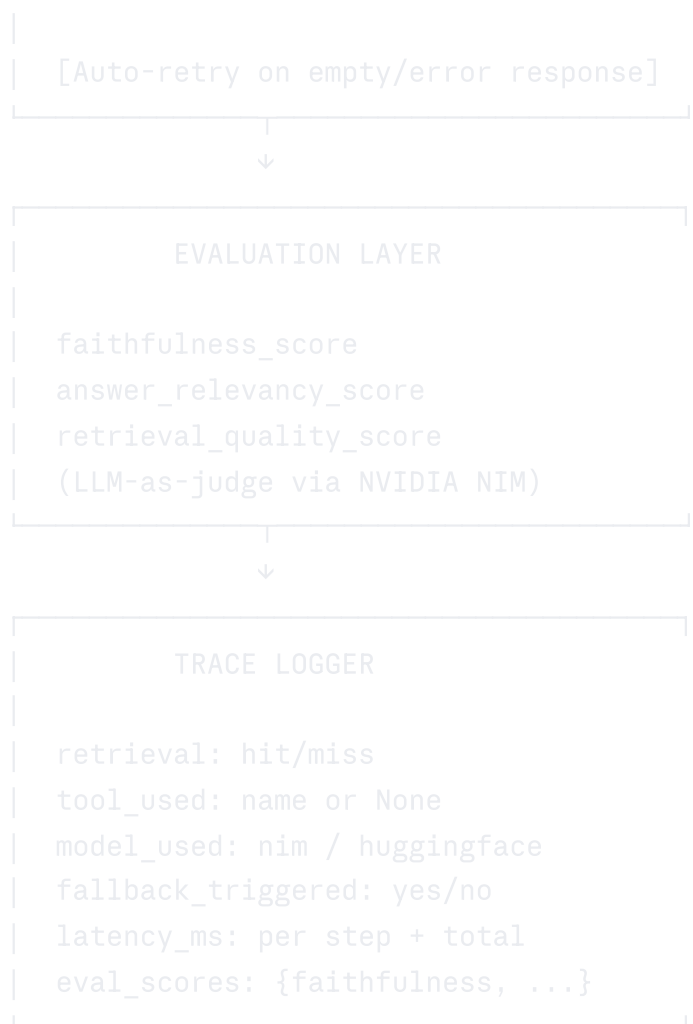
Day	Focus	What You'll Learn
1	Data extraction + Milvus loading	PDF parsing, web scraping, embeddings, vector DB
2	Tool definitions + Worker agents	Function/tool calling, agentic patterns
3	Supervisor agent + Fallback LLM	Orchestration, supervisor-worker pattern
4	FastAPI (/load + /query) + Tracer	API design, structured logging, observability
5	Evaluation module	RAG evaluation metrics, LLM-as-judge

Day	Focus	What You'll Learn
6	Streamlit UI + README + GitHub	Full-stack integration, documentation

Day 7 = Buffer. Polish, re-test edge cases, record a 2-min demo video (interviewers love this).

Architecture





Key Concepts You Will Learn (Interview-Ready)

1. Tool Calling / Function Calling

Tools are **Python functions with defined schemas** that the agent can choose to invoke. The LLM doesn't run code — it decides *which tool to call and with what arguments*, and your code executes it.

python

```
# Example tool definition (tools.py)
def web_search_tool(query: str) -> str:
    """Search the web for current information not in the knowledge base."""
    ...

def calculator_tool(expression: str) -> str:
    """Evaluate a mathematical expression."""
    ...

def sentiment_tool(text: str) -> str:
    """Return sentiment label and score for a piece of review text."""
    ...

TOOLS = {
    "web_search": web_search_tool,
    "calculator": calculator_tool,
    "sentiment_analysis": sentiment_tool,
}
```

Interview answer: "Function calling means the LLM outputs a structured JSON like `{tool: 'web_search', args: {query: '...'}}` and our orchestration layer executes the actual function. The LLM never directly calls APIs — it just decides."

2. Supervisor-Worker Pattern

The **Supervisor** is a higher-level agent that receives the query and makes a routing decision. It doesn't answer questions itself — it *delegates* to the right worker.

```
Query → Supervisor → [Decision] → RAG Worker → LLM → Answer
                                → Tool Worker → LLM → Answer
```

Why this matters: In production systems (like real MAS at AI companies), no single agent does everything. The supervisor keeps logic clean and each worker is independently testable. This is what you'll see at scale.

3. Agentic Loop / Orchestration

The agent doesn't just call an LLM once. It runs in a **loop**:

1. Observe (what's the query, what's in context?)
2. Decide (which path?)
3. Act (retrieve / call tool)
4. Generate (LLM)

- 5. Evaluate (are we done? is the answer good enough?)
- 6. Return or retry

4. Tracing / Observability

Every step in the loop is logged with timestamps so you can debug and explain *why* the system made the decisions it did. This is what Langfuse does in production. You'll build a lightweight version.

5. Evaluation (RAG Evaluation)

After generating an answer, you run a second pass to *score it* using another LLM call. Key metrics:

Metric	Question It Answers
Faithfulness	Does the answer only use what's in the retrieved context? (no hallucination)
Answer Relevancy	Is the answer actually relevant to the question asked?
Context Recall	Did we retrieve chunks that actually help answer this question?
Retrieval Score	Raw cosine similarity from Milvus — quantitative proxy for retrieval quality

Project Structure

```
productmind/
├── README.md
├── requirements.txt
├── .env.example
├──
├── core/
│   ├── extractor.py           # PDF + URL text extraction
│   ├── chunker.py            # Text chunking + preprocessing
│   ├── embedder.py           # SentenceTransformers embeddings
│   └── vector_store.py        # Milvus CRUD operations
├──
├── agent/
│   ├── supervisor.py         # Supervisor: routing decision logic
│   ├── rag_worker.py         # RAG Worker: retrieval + context building
│   ├── tool_worker.py        # Tool Worker: tool selection + execution
│   └── tools.py               # Tool definitions (web search, calculator,
```

```
sentiment)
|   └─ llm_client.py          # NVIDIA NIM primary + HuggingFace fallback
|
|─ evaluation/
|   └─ evaluator.py          # Evaluation metrics (faithfulness, relevancy,
context recall)
|   └─ eval_prompts.py       # Prompt templates for LLM-as-judge scoring
|
|─ tracing/
|   └─ tracer.py             # Structured step-by-step trace logger (JSON +
console)
|
|─ api/
|   └─ main.py               # FastAPI app – /load and /query endpoints
|
|─ ui/
|   └─ app.py                # Streamlit frontend
|
└─ logs/
    └─ traces/               # JSON trace files, one per query
```

Detailed Step-by-Step Implementation

Day 1 — Data Extraction + Vector DB Loading

core/extractor.py

```
python
```

```

import pdfplumber
import requests
from bs4 import BeautifulSoup
from typing import Union
import logging

logger = logging.getLogger(__name__)

def extract_from_pdf(file_path: str) -> str:
    """Extract text from PDF using pdfplumber."""
    text = ""
    with pdfplumber.open(file_path) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
    logger.info(f"Extracted {len(text)} characters from PDF: {file_path}")
    return text

def extract_from_url(url: str) -> str:
    """Scrape visible text from a webpage using BeautifulSoup."""
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    soup = BeautifulSoup(response.text, "html.parser")
    # Remove script/style tags
    for tag in soup(["script", "style", "nav", "footer"]):
        tag.decompose()
    text = soup.get_text(separator="\n")
    # Clean up whitespace
    lines = [line.strip() for line in text.splitlines() if line.strip()]
    clean_text = "\n".join(lines)
    logger.info(f"Extracted {len(clean_text)} characters from URL: {url}")
    return clean_text

def extract(source: str, source_type: str) -> str:
    """Unified extractor – routes to PDF or URL extractor."""
    if source_type == "pdf":
        return extract_from_pdf(source)
    elif source_type == "url":
        return extract_from_url(source)
    else:
        raise ValueError(f"Unsupported source type: {source_type}")

```

core/chunker.py

python

```
from typing import List
import re

def chunk_text(text: str, chunk_size: int = 500, overlap: int = 50) -> List[str]:
    """
    Split text into overlapping chunks.

    chunk_size: target character count per chunk
    overlap: characters shared between consecutive chunks (preserves context)
    """
    # Clean text
    text = re.sub(r'\n{3,}', '\n\n', text) # collapse multiple newlines
    text = text.strip()

    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        # Try to break at a sentence boundary
        if end < len(text):
            last_period = text.rfind('.', start, end)
            if last_period > start + chunk_size // 2:
                end = last_period + 1
        chunks.append(text[start:end].strip())
        start = end - overlap # overlap for context continuity

    return [c for c in chunks if len(c) > 50] # filter out tiny fragments
```

core/embedder.py

python

```
from sentence_transformers import SentenceTransformer
from typing import List
import numpy as np

MODEL_NAME = "all-MiniLM-L6-v2" # Fast, 384-dim, free
_model = None

def get_model() -> SentenceTransformer:
    global _model
    if _model is None:
        _model = SentenceTransformer(MODEL_NAME)
    return _model

def embed(texts: List[str]) -> List[List[float]]:
    """Generate embeddings for a list of text chunks."""
    model = get_model()
    embeddings = model.encode(texts, normalize_embeddings=True)
    return embeddings.tolist()

def embed_query(query: str) -> List[float]:
    """Generate embedding for a single query string."""
    return embed([query])[0]
```

core/vector_store.py

python

```

from pymilvus import connections, Collection, FieldSchema, CollectionSchema, Da
from typing import List, Dict, Any
import logging

logger = logging.getLogger(__name__)

COLLECTION_NAME = "productmind_kb"
EMBEDDING_DIM = 384 # all-MiniLM-L6-v2 dimension
SIMILARITY_THRESHOLD = 0.5 # Cosine similarity cutoff - tune this

def connect():
    connections.connect(host="localhost", port="19530")

def ensure_collection():
    """Create Milvus collection if it doesn't exist."""
    if utility.has_collection(COLLECTION_NAME):
        return Collection(COLLECTION_NAME)

    fields = [
        FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
        FieldSchema(name="source", dtype=DataType.VARCHAR, max_length=500),
        FieldSchema(name="chunk_text", dtype=DataType.VARCHAR, max_length=2000),
        FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=EMBEDDING_DIM)
    ]
    schema = CollectionSchema(fields, description="ProductMind Knowledge Base")
    collection = Collection(COLLECTION_NAME, schema)

    # Create IVF_FLAT index for cosine similarity
    index_params = {"metric_type": "COSINE", "index_type": "IVF_FLAT", "params": {"m": 8, "nlist": 256}}
    collection.create_index("embedding", index_params)
    logger.info(f"Created Milvus collection: {COLLECTION_NAME}")
    return collection

def insert_chunks(chunks: List[str], embeddings: List[List[float]], source: str):
    """Insert text chunks and their embeddings into Milvus."""
    connect()
    collection = ensure_collection()
    data = [
        [source] * len(chunks), # source field
        chunks,                  # chunk_text field
        embeddings,               # embedding field
    ]
    collection.insert(data)
    collection.flush()
    logger.info(f"Inserted {len(chunks)} chunks from {source}")

```

```
def search(query_embedding: List[float], top_k: int = 5) -> List[Dict[str, Any]]:
    """
    Search Milvus for nearest chunks to query embedding.
    Returns list of dicts with chunk_text and similarity score.
    """
    connect()
    collection = ensure_collection()
    collection.load()

    results = collection.search(
        data=[query_embedding],
        anns_field="embedding",
        param={"metric_type": "COSINE", "params": {"nprobe": 10}},
        limit=top_k,
        output_fields=["chunk_text", "source"],
    )

    hits = []
    for hit in results[0]:
        hits.append({
            "chunk_text": hit.entity.get("chunk_text"),
            "source": hit.entity.get("source"),
            "score": hit.score,
        })
    return hits
```

Day 2 — Tool Definitions + Worker Agents

agent/tools.py

python

```

import os
import requests
import math
from typing import Any

# -- Web Search Tool --
def web_search_tool(query: str) -> str:
    """
    Search the web using DuckDuckGo Instant Answer API (free, no key needed).
    Falls back to a simple message if no result found.
    """
    try:
        url = f"https://api.duckduckgo.com/?q={query}&format=json&no_html=1&ski
        resp = requests.get(url, timeout=5)
        data = resp.json()
        abstract = data.get("AbstractText", "")
        if abstract:
            return abstract
        # Try related topics
        topics = data.get("RelatedTopics", [])
        if topics and isinstance(topics[0], dict):
            return topics[0].get("Text", f"No result found for: {query}")
        return f"No relevant web result found for: {query}"
    except Exception as e:
        return f"Web search failed: {str(e)}"

# -- Calculator Tool --
def calculator_tool(expression: str) -> str:
    """Safely evaluate a math expression."""
    try:
        allowed = set('0123456789+-*/( )., ')
        if not all(c in allowed for c in expression):
            return "Invalid characters in expression."
        result = eval(expression, {"__builtins__": {}}, {"math": math})
        return str(result)
    except Exception as e:
        return f"Calculation error: {str(e)}"

# -- Sentiment Tool --
def sentiment_tool(text: str) -> str:
    """
    Simple rule-based sentiment scoring.
    In production, you'd call a dedicated model.
    """
    positive_words = {"good", "great", "excellent", "amazing", "love", "best",
    negative_words = {"bad", "poor", "terrible", "awful", "hate", "worst", "bro

```

```

words = set(text.lower().split())
pos = len(words & positive_words)
neg = len(words & negative_words)

if pos > neg:
    sentiment = "POSITIVE"
elif neg > pos:
    sentiment = "NEGATIVE"
else:
    sentiment = "NEUTRAL"

    return f"Sentiment: {sentiment} (positive signals: {pos}, negative signals: {neg})"

# Tool registry – the supervisor picks from this
TOOL_REGISTRY = {
    "web_search": {
        "fn": web_search_tool,
        "description": "Search the web for current or factual information not in the knowledge base",
        "args": ["query"],
    },
    "calculator": {
        "fn": calculator_tool,
        "description": "Evaluate mathematical expressions or perform numeric calculations",
        "args": ["expression"],
    },
    "sentiment_analysis": {
        "fn": sentiment_tool,
        "description": "Analyse sentiment of a piece of review or customer feedback",
        "args": ["text"],
    },
}

def execute_tool(tool_name: str, **kwargs) -> str:
    """Execute a tool by name with provided arguments."""
    if tool_name not in TOOL_REGISTRY:
        return f"Unknown tool: {tool_name}"
    return TOOL_REGISTRY[tool_name]["fn"](**kwargs)

```

agent/rag_worker.py

python

```

from core.embedder import embed_query
from core.vector_store import search, SIMILARITY_THRESHOLD
from typing import Dict, Any, List

def retrieve(query: str, top_k: int = 5) -> Dict[str, Any]:
    """
    Retrieve relevant context from Milvus.
    Returns: {hit: bool, context: str, score: float, chunks: list}
    """
    query_embedding = embed_query(query)
    results = search(query_embedding, top_k=top_k)

    if not results or results[0]["score"] < SIMILARITY_THRESHOLD:
        return {"hit": False, "context": "", "score": results[0]["score"] if re

    # Combine top chunks into a single context block
    context = "\n\n---\n\n".join([r["chunk_text"] for r in results])
    return {
        "hit": True,
        "context": context,
        "score": results[0]["score"],
        "chunks": results,
    }

def build_rag_prompt(query: str, context: str) -> str:
    return f"""You are a helpful product intelligence assistant.
    Answer the user's question using ONLY the context provided below.
    If the context doesn't contain enough information, say so clearly.

    CONTEXT:
    {context}

    QUESTION: {query}

    ANSWER: """

```

agent/tool_worker.py

python

```

from agent.tools import TOOL_REGISTRY, execute_tool
from typing import Dict, Any

def select_tool(query: str, llm_client) -> str:
    """
    Ask the LLM to pick the right tool for the query.
    Returns the tool name as a string.
    """
    tool_list = "\n".join([
        f"- {name}: {info['description']}"
        for name, info in TOOL_REGISTRY.items()
    ])

    prompt = f"""You are a tool selector. Given the user query, pick the SINGLE

Available tools:
{tool_list}

Query: {query}

Respond with ONLY the tool name (exactly as listed). Nothing else."""

    tool_name = llm_client.generate(prompt).strip().lower()

    # Validate - default to web_search if invalid
    if tool_name not in TOOL_REGISTRY:
        tool_name = "web_search"
    return tool_name

def run_tool_worker(query: str, llm_client) -> Dict[str, Any]:
    """Run tool selection + execution and return result."""
    tool_name = select_tool(query, llm_client)

    # Build the right argument based on tool
    if tool_name == "web_search":
        result = execute_tool("web_search", query=query)
    elif tool_name == "calculator":
        result = execute_tool("calculator", expression=query)
    elif tool_name == "sentiment_analysis":
        result = execute_tool("sentiment_analysis", text=query)
    else:
        result = execute_tool("web_search", query=query)

    return {"tool_used": tool_name, "tool_output": result}

def build_tool_prompt(query: str, tool_output: str) -> str:

```

```
return f"""You are a helpful assistant. Use the tool output below to answer
```

```
TOOL OUTPUT:  
{tool_output}
```

```
QUESTION: {query}
```

```
ANSWER: """
```

Day 3 — Supervisor + LLM Client with Fallback

agent/llm_client.py

```
python
```

```

import os
import time
import logging
from openai import OpenAI # NVIDIA NIM uses OpenAI-compatible API

logger = logging.getLogger(__name__)

# NVIDIA NIM config
NIM_BASE_URL = "https://integrate.api.nvidia.com/v1"
NIM_MODEL = "meta/llama-3.1-8b-instruct" # or llama-3.3-70b-instruct

# HuggingFace fallback (using Inference API, free tier)
HF_API_URL = "https://api-inference.huggingface.co/models/mistralai/Mistral-7B-

class LLMClient:
    def __init__(self):
        self.nim_client = OpenAI(
            base_url=NIM_BASE_URL,
            api_key=os.environ.get("NVIDIA_API_KEY"),
        )
        self.hf_token = os.environ.get("HF_API_TOKEN")

    def generate(self, prompt: str, max_tokens: int = 512) -> str:
        """Try NVIDIA NIM first. Fall back to HuggingFace on failure."""
        response, model_used, fallback_triggered = self._try_nim(prompt, max_to

        if not response:
            logger.warning("NIM returned empty/failed. Triggering HuggingFace f
            response, fallback_triggered = self._try_huggingface(prompt), True
            model_used = "huggingface_mistral"

        return response, model_used, fallback_triggered

    def _try_nim(self, prompt: str, max_tokens: int):
        """Call NVIDIA NIM. Returns (response_text, model_name, fallback_trigge
        try:
            completion = self.nim_client.chat.completions.create(
                model=NIM_MODEL,
                messages=[{"role": "user", "content": prompt}],
                max_tokens=max_tokens,
                temperature=0.3,
            )
            text = completion.choices[0].message.content.strip()
            if not text:
                return None, NIM_MODEL, False
            return text, NIM_MODEL, False

```

```

except Exception as e:
    logger.error(f"NIM call failed: {e}")
    return None, NIM_MODEL, False

def _try_huggingface(self, prompt: str) -> str:
    """Call HuggingFace Inference API as fallback."""
    import requests
    headers = {"Authorization": f"Bearer {self.hf_token}"}
    payload = {"inputs": prompt, "parameters": {"max_new_tokens": 300}}
    try:
        resp = requests.post(HF_API_URL, headers=headers, json=payload, timeout=30)
        result = resp.json()
        if isinstance(result, list) and result:
            generated = result[0].get("generated_text", "")
            return generated.replace(prompt, "").strip()
        return "Fallback model could not generate a response."
    except Exception as e:
        logger.error(f"HuggingFace fallback also failed: {e}")
        return "Both primary and fallback models failed. Please try again."

```

agent/supervisor.py

python

```

import time
import logging
from agent.rag_worker import retrieve, build_rag_prompt
from agent.tool_worker import run_tool_worker, build_tool_prompt
from agent.llm_client import LLMClient
from tracing.tracer import Tracer
from evaluation.evaluator import evaluate_response

logger = logging.getLogger(__name__)

class SupervisorAgent:
    """
    The Supervisor makes the routing decision:
    - If Milvus has relevant context (score >= threshold) → RAG Worker
    - If not → Tool Worker

    Then calls the LLM (with fallback) and runs evaluation.
    """

    def __init__(self):
        self.llm = LLMClient()
        self.tracer = Tracer()

    def run(self, query: str) -> dict:
        trace = self.tracer.start(query)
        start_total = time.time()

        # Step 1: Check vector DB
        t0 = time.time()
        retrieval_result = retrieve(query)
        retrieval_latency = round((time.time() - t0) * 1000, 2)

        trace.log_step("retrieval", {
            "hit": retrieval_result["hit"],
            "score": retrieval_result["score"],
            "latency_ms": retrieval_latency,
        })

        # Step 2: Route to correct worker
        tool_used = None
        if retrieval_result["hit"]:
            # RAG path
            prompt = build_rag_prompt(query, retrieval_result["context"])
            path = "rag"
            context_for_eval = retrieval_result["context"]
        else:

```

```

# Tool path
t1 = time.time()
tool_result = run_tool_worker(query, self.llm)
tool_latency = round((time.time() - t1) * 1000, 2)
tool_used = tool_result["tool_used"]

trace.log_step("tool_call", {
    "tool": tool_used,
    "latency_ms": tool_latency,
})

prompt = build_tool_prompt(query, tool_result["tool_output"])
path = "tool"
context_for_eval = tool_result["tool_output"]

# Step 3: Generate answer (with fallback)
t2 = time.time()
answer, model_used, fallback_triggered = self.llm.generate(prompt)
llm_latency = round((time.time() - t2) * 1000, 2)

trace.log_step("llm_generation", {
    "model": model_used,
    "fallback": fallback_triggered,
    "latency_ms": llm_latency,
})

# Step 4: Evaluate the response
t3 = time.time()
eval_scores = evaluate_response(query, answer, context_for_eval, self.llm)
eval_latency = round((time.time() - t3) * 1000, 2)

trace.log_step("evaluation", {
    "scores": eval_scores,
    "latency_ms": eval_latency,
})

total_latency = round((time.time() - start_total) * 1000, 2)

# Final trace summary
final_trace = trace.finish({
    "path": path,
    "retrieval_hit": retrieval_result["hit"],
    "retrieval_score": retrieval_result["score"],
    "tool_used": tool_used,
    "model_used": model_used,
    "fallback_triggered": fallback_triggered,
    "eval_scores": eval_scores,
})

```

```
        "total_latency_ms": total_latency,
    })

    return {"answer": answer, "trace": final_trace}
```

Day 4 — Tracer + FastAPI

tracing/tracer.py

python

```

import json
import uuid
import time
import logging
from datetime import datetime
from pathlib import Path

LOG_DIR = Path("logs/traces")
LOG_DIR.mkdir(parents=True, exist_ok=True)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
)

class QueryTrace:
    def __init__(self, query: str):
        self.trace_id = str(uuid.uuid4())[:8]
        self.query = query
        self.started_at = datetime.utcnow().isoformat()
        self.steps = []

    def log_step(self, step_name: str, data: dict):
        self.steps.append({"step": step_name, "timestamp": datetime.utcnow().isoformat()})
        logging.info(f"[{self.trace_id}] STEP={step_name} | {data}")

    def finish(self, summary: dict) -> dict:
        trace = {
            "trace_id": self.trace_id,
            "query": self.query,
            "started_at": self.started_at,
            "finished_at": datetime.utcnow().isoformat(),
            "steps": self.steps,
            "summary": summary,
        }
        # Persist trace to JSON file
        trace_file = LOG_DIR / f"{self.trace_id}.json"
        trace_file.write_text(json.dumps(trace, indent=2))
        return trace

class Tracer:
    def start(self, query: str) -> QueryTrace:
        return QueryTrace(query)

```

python

```

import os
import shutil
import tempfile
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from pydantic import BaseModel, HttpUrl
from core.extractor import extract
from core.chunker import chunk_text
from core.embedder import embed
from core.vector_store import insert_chunks
from agent.supervisor import SupervisorAgent

app = FastAPI(title="ProductMind API", version="1.0.0")
supervisor = SupervisorAgent()

# ----- /load endpoint -----

class LoadURLRequest(BaseModel):
    url: str

@app.post("/load/url")
async def load_from_url(request: LoadURLRequest):
    """Ingest text from a URL into Milvus."""
    try:
        text = extract(request.url, "url")
        if not text.strip():
            raise HTTPException(status_code=400, detail="No text content extracted")
        chunks = chunk_text(text)
        embeddings = embed(chunks)
        insert_chunks(chunks, embeddings, source=request.url)
        return {"status": "ok", "source": request.url, "chunks_stored": len(chunks)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/load/file")
async def load_from_file(file: UploadFile = File(...)):
    """Ingest text from an uploaded PDF file into Milvus."""
    if not file.filename.endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are supported")
    try:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
            shutil.copyfileobj(file.file, tmp)
            tmp_path = tmp.name

        text = extract(tmp_path, "pdf")
        os.unlink(tmp_path)

```

```

        if not text.strip():
            raise HTTPException(status_code=400, detail="No text content extracted")

        chunks = chunk_text(text)
        embeddings = embed(chunks)
        insert_chunks(chunks, embeddings, source=file.filename)
        return {"status": "ok", "source": file.filename, "chunks_stored": len(chunks)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ----- /query endpoint -----

class QueryRequest(BaseModel):
    query: str

@app.post("/query")
async def query(request: QueryRequest):
    """Run agentic RAG query. Returns answer + full trace."""
    if not request.query.strip():
        raise HTTPException(status_code=400, detail="Query cannot be empty.")
    try:
        result = supervisor.run(request.query)
        return {"answer": result["answer"], "trace": result["trace"]}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def health():
    return {"status": "ok"}

```

Day 5 — Evaluation Module

evaluation/eval_prompts.py

python

```
FAITHFULNESS_PROMPT = """You are an evaluation judge.
```

```
Question: {question}
```

```
Context: {context}
```

```
Answer: {answer}
```

```
Score the answer's FAITHFULNESS to the context on a scale of 0.0 to 1.0.
```

- 1.0 = Every claim in the answer is supported by the context
- 0.5 = Some claims are supported, some are not
- 0.0 = The answer ignores the context or contradicts it

```
Respond ONLY with a number between 0.0 and 1.0. Nothing else."""
```

```
RELEVANCY_PROMPT = """You are an evaluation judge.
```

```
Question: {question}
```

```
Answer: {answer}
```

```
Score the answer's RELEVANCY to the question on a scale of 0.0 to 1.0.
```

- 1.0 = The answer directly and completely addresses the question
- 0.5 = The answer partially addresses the question
- 0.0 = The answer does not address the question

```
Respond ONLY with a number between 0.0 and 1.0. Nothing else."""
```

evaluation/evaluator.py

```
python
```

```

import logging
from evaluation.eval_prompts import FAITHFULNESS_PROMPT, RELEVANCY_PROMPT

logger = logging.getLogger(__name__)

def safe_score(raw: str) -> float:
    """Parse a float score from LLM output. Defaults to 0.5 on parse failure."""
    try:
        val = float(raw.strip().split()[0])
        return round(max(0.0, min(1.0, val)), 3)
    except:
        return 0.5

def evaluate_response(question: str, answer: str, context: str, llm_client) -> dict:
    """
    Run LLM-as-judge evaluation.
    Returns dict of metric scores.
    """
    scores = {}

    # Faithfulness
    faith_prompt = FAITHFULNESS_PROMPT.format(
        question=question, context=context[:2000], answer=answer
    )
    raw_faith, _, _ = llm_client.generate(faith_prompt, max_tokens=10)
    scores["faithfulness"] = safe_score(raw_faith)

    # Answer Relevancy
    rel_prompt = RELEVANCY_PROMPT.format(question=question, answer=answer)
    raw_rel, _, _ = llm_client.generate(rel_prompt, max_tokens=10)
    scores["answer_relevancy"] = safe_score(raw_rel)

    # Composite score
    scores["composite"] = round((scores["faithfulness"] + scores["answer_relevancy"]) / 2, 3)

    logger.info(f"Eval scores: {scores}")
    return scores

```

Day 6 — Streamlit UI

ui/app.py

python

```

import streamlit as st
import requests
import json

API_BASE = "http://localhost:8000"

st.set_page_config(page_title="ProductMind", page_icon="🧠", layout="wide")
st.title("🧠 ProductMind – Agentic Product Intelligence")
st.caption("Upload product documentation, then ask anything.")

# ----- Sidebar: Load Sources -----
with st.sidebar:
    st.header("📁 Load Knowledge Base")

    load_type = st.radio("Source type", ["PDF File", "URL"])

    if load_type == "PDF File":
        uploaded_file = st.file_uploader("Upload PDF", type=["pdf"])
        if st.button("Load PDF") and uploaded_file:
            with st.spinner("Extracting and embedding..."):
                response = requests.post(
                    f"{API_BASE}/load/file",
                    files={"file": (uploaded_file.name, uploaded_file.getvalue())}
                )
            if response.ok:
                data = response.json()
                st.success(f"✅ Loaded {data['chunks_stored']} chunks from {data['source']}")
            else:
                st.error(f"❌ {response.json().get('detail', 'Failed')}")

    else:
        url_input = st.text_input("Enter URL")
        if st.button("Load URL") and url_input:
            with st.spinner("Scraping and embedding..."):
                response = requests.post(
                    f"{API_BASE}/load/url",
                    json={"url": url_input}
                )
            if response.ok:
                data = response.json()
                st.success(f"✅ Loaded {data['chunks_stored']} chunks from URL")
            else:
                st.error(f"❌ {response.json().get('detail', 'Failed')}")

# ----- Main: Query Interface -----
query = st.text_input("💬 Ask a question", placeholder="e.g. What are the top c")

```

```

if st.button("Ask", type="primary") and query:
    with st.spinner("Thinking..."):
        response = requests.post(f"{API_BASE}/query", json={"query": query})

    if response.ok:
        result = response.json()
        answer = result["answer"]
        trace = result["trace"]
        summary = trace.get("summary", {})

        # Answer
        st.markdown("### 💡 Answer")
        st.write(answer)

        # Trace panel
        st.markdown("----")
        st.markdown("### 🔍 Trace & Observability")

        col1, col2, col3 = st.columns(3)

        with col1:
            st.markdown("**Retrieval**")
            hit = summary.get("retrieval_hit", False)
            score = summary.get("retrieval_score", 0)
            st.metric("Status", "✅ HIT" if hit else "❌ MISS")
            st.metric("Similarity Score", f"{score:.3f}")

        with col2:
            st.markdown("**Routing**")
            path = summary.get("path", "unknown").upper()
            tool = summary.get("tool_used") or "-"
            model = summary.get("model_used", "unknown")
            fallback = "Yes ⚠️" if summary.get("fallback_triggered") else "No ✅"
            st.metric("Path", path)
            st.metric("Tool Used", tool)
            st.metric("Model", model)
            st.metric("Fallback", fallback)

        with col3:
            st.markdown("**Evaluation**")
            eval_scores = summary.get("eval_scores", {})
            st.metric("Faithfulness", f"{eval_scores.get('faithfulness', 0):.2f}")
            st.metric("Answer Relevancy", f"{eval_scores.get('answer_relevancy', 0):.2f}")
            st.metric("Composite Score", f"{eval_scores.get('composite', 0):.2f}")
            st.metric("Total Latency", f"{summary.get('total_latency_ms', 0)} m

```

```
# Step-by-step log (expandable)
with st.expander("📄 Step-by-step trace"):
    for step in trace.get("steps", []):
        st.json(step)

else:
    st.error(f"Query failed: {response.json().get('detail', 'Unknown error')}
```

Evaluation: Interview Q&A Prep

These are real questions you may be asked. Study the answers because you built this.

Q: What's the difference between RAG and fine-tuning?

Fine-tuning bakes knowledge into model weights — expensive, static. RAG keeps knowledge external in a vector DB — cheap, updateable. You use RAG when knowledge changes frequently (product updates, policies, reviews). Fine-tuning when you need consistent *behaviour* (tone, format, domain-specific reasoning style).

Q: How does your supervisor know when to use RAG vs a tool?

It's threshold-based: we compute cosine similarity between the query embedding and all stored chunks in Milvus. If the best match is below 0.5 (tunable), it means the knowledge base doesn't have relevant information, so we route to the tool worker. Above the threshold, RAG can answer.

Q: What is function calling?

Function calling is when you give the LLM a schema of available functions (name, description, parameters) and it returns a structured JSON specifying which function to call and with what arguments. Your code then executes it. The LLM never runs code — it only decides.

Q: How do you prevent hallucination in a RAG system?

Three ways: (1) Prompt the LLM to only use the provided context — "answer only from the context below, if unsure say so". (2) Set a retrieval threshold — if no relevant context is found, don't attempt RAG at all. (3) Run a faithfulness evaluation after generation to score whether the answer is grounded in the retrieved chunks.

Q: What is LLM-as-judge evaluation?

You use a separate LLM call to score the quality of the generated answer. You give it the question, context, and answer, and ask it to return a numeric score. It's scalable (no human needed) and works well for faithfulness and relevancy metrics. The weakness is the judge LLM can also hallucinate, so you calibrate against human judgements.

Q: What is the supervisor-worker pattern?

The supervisor is a higher-level orchestrator that receives the task and decides *who* should handle it and *how*. Workers are specialised and focused — RAG worker only does retrieval, tool worker only calls external APIs. This keeps each component testable, replaceable, and single-responsibility. It directly maps to multi-agent systems in production.

Q: Why Milvus over alternatives like FAISS or Chroma?

FAISS is in-memory and not persistent. Chroma is great for prototypes but doesn't scale to production. Milvus is a dedicated vector database — persistent, supports multi-tenancy, horizontal scaling, and has native support for cosine/L2 similarity with ANN indexing (IVF_FLAT, HNSW). Production AI systems use Milvus or Pinecone.

README Structure (for GitHub)

```
# ProductMind – Agentic RAG System for Product Intelligence

## Overview
A multi-agent RAG system using supervisor-worker architecture, built with
LangChain, Milvus, NVIDIA NIM, and FastAPI.

## Architecture
[include the ASCII diagram from above]

## Setup
1. Clone repo
2. Install: `pip install -r requirements.txt`
3. Start Milvus: `docker-compose up -d milvus`
4. Copy `.env.example` → `.env` and add your API keys
5. Run API: `uvicorn api.main:app --reload`
6. Run UI: `streamlit run ui/app.py`

## Endpoints
- POST /load/file – ingest PDF
- POST /load/url – ingest webpage
- POST /query – agentic query with trace

## Key Design Decisions
- Supervisor routes by similarity threshold (tunable in vector_store.py)
- NVIDIA NIM (Llama-3) primary | HuggingFace Mistral-7B fallback
- LLM-as-judge evaluation for faithfulness + relevancy
- Every query produces a JSON trace file in logs/traces/
```

Your Unique Differentiator When Interviewed

Most candidates building RAG projects treat it as a pipeline. You have something extra:

"I've actually seen this problem from the business side. At Capchek, I designed dashboards that showed brand teams *why* customers were unhappy — support ticket trends, issue concentration, resolution times. ProductMind is what I'd build on top of that analytics layer: an intelligent agent that doesn't just visualise the data, but answers questions from it. The evaluation module is something I added because, in my analytics work, every metric had to be justified to stakeholders — I applied the same thinking to the AI layer."

That story connects your analytics background to the AI work. Interviewers remember stories.